Moreover, we decided to implement a check for elements belonging to different partitions rather than for them to belong to the same one: this is because of how disjoint sets are normally used: we are normally interested in checking if two elements don't belong to the same partition, and if that's the case we merge the two partitions. But depending on the use you are going to make of this container, it is possible that the other way around is more convenient, and there is nothing preventing you to define a `samePartition` method instead.

All the methods we have seen so far run in constant time with respect to the size of the container. Now, it's time to implement the method merging two partitions, shown in listing 5.5 (and illustrated in figure 5.5). As we have seen, the `merge` method requires `O(n)` assignments in the worst case.

Listing 5.5 Naïve solution, merge

```
function merge(elem1,
elem2)                                                    #1
  p1 = this.findPartition(elem1)
                                          #2
  p2 =
this.findPartition(elem2)
         #3
  if p1 == p2
then
#4
    return false
  for each elem in
p1                                                          #5

p2.add(elem)
```